Machine Learning for Mechanical Engineers at CoEP Session 12: Ensemble RF

Session 12: Ensemble

Ensemble

For Complex Decision Making

- How do I handle data for large number of variables?
- More complex the decision making, more experts are needed.
- Linux, for example, is such a complex system that building it took hundreds of experts.
- Ensemble means collection of models, meaning collection of experts!!

What is Ensemble?

- Definition: An ensemble is a set of elements that collectively contribute to a whole.
- A familiar example is a musical ensemble, which blends the sounds of several musical instruments to create a beautiful harmony, or architectural ensembles, which are a set of buildings designed as a unit.
- In ensembles, the (whole) harmonious outcome is more important than the performance of any individual part.

Theorem about Ensemble

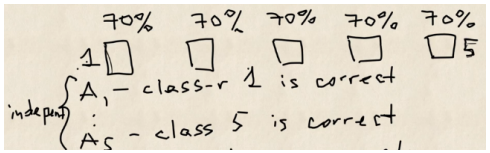
- Condorcet's jury theorem (1784) is about an ensemble in some sense.
- It states that, if each member of the jury makes an independent judgment and the probability of the correct decision by each juror is more than 0.5, then the probability of the correct decision by the whole jury increases with the total number of jurors and tends to one.
- On the other hand, if the probability of being right is less than 0.5 for each juror, then the probability of the correct decision by the whole jury decreases with the number of jurors and tends to zero.

Idea behind Ensemble

- Rational: 'No Free Lunch' Theorem: Even popular base classifiers will perform poorly on some datasets, where the learning classifier and data distribution do not match well
- Intuitive Justification: When combining multiple, independent, and diverse, decisions each of which is at least more accurate than random guessing then random errors cancel each other out, and correct decisions are reinforced

Idea behind Ensemble

- 5 classifiers each with 0.7 accuracy.
- All of them give predictions.
- Some match , some don't.
- Majority voting means >= 3 should be correct. So when all 5 or 4 or 3 are correct.



Idea behind Ensemble

- Probability that all 5 classifiers would be correct is $(0.7) * (0.7) \ldots = (0.7)^5$
- Probability that 4 classifiers would be correct and one incorrect is $(0.7)^4 \times (0.3)^1$
- Binomial theorem says that there are $\binom{5}{1}$ arrangements of this possible. See, the incorrect one can be any one of the 5. So the probability is $\binom{5}{1}(0.7)^4 \times (0.3)^1$
- Probability that 3 classifiers would be correct and two incorrect is $\binom{5}{2}(0.7)^3 \times (0.3)^2$
- Total probability is to add them up $= (0.7)^5 + \binom{5}{1}(0.7)^4 \times (0.3)^1 + \binom{5}{2}(0.7)^3 \times (0.3)^2 \approx 0.84$

Did you see? Individually they were 0.7 each, but together 0.84!!! If we take $N = 1000$ classifiers, then the total accuracy goes to 0.99

Jury to Ensemble?

$\mu = \sum_{i=m}^{N} \binom{N}{i} p^i (1-p)^{N-i}$
Where,

- N is the total number of jurors;
- m is a minimal number of jurors that would make a majority, that is $m = \frac{N+1}{2}$;
- $\binom{N}{i}$ is the number of i-combinations from a set with N elements.
- p is the probability of the correct decision by a juror;
- μ is the probability of the correct decision by the whole jury.

It can be seen that if $p > 0.5$, then $\mu > p$. In addition, if $N \to \infty$, then $\mu \to 1$.

Intuition
Think of each juror's mistake as an independent coin flip biased slightly towards being correct. Pooling many such flips through a majority vote makes the rare event "more than half are simultaneously wrong" vanishingly unlikely – exactly like how averaging many noisy measurements cancels out random error. This only works because the mistakes are independent; correlated experts who share the same blind spot don't get this benefit.

Wisdom of the crowd

- In 1906, Francis Galton visited a country fair in Plymouth where he saw a contest being held for farmers.
- 800 participants tried to estimate the weight of a slaughtered bull.
- The real weight of the bull was 1198 pounds. Although none of the farmers could guess the exact weight of the animal, the average of their predictions was 1197 pounds.

A similar idea for error reduction is adopted in the field of Machine Learning, called Ensemble.

Ensemble Rationale

Another Example:

- Assume we have 25 binary classifiers
- Each has error rate: $\epsilon = 0.35$
- If all 25 classifiers are identical:
- They will vote the same way on each test instance.
- Majority wins, but since every classifier agrees, the ensemble simply inherits the same error rate: $\epsilon = 0.35$ – no improvement at all.

Ensemble Rationale

- Ensemble method only makes a wrong prediction if more than half of the base classifiers predict incorrectly.
- The probability that the ensemble classifier make's a wrong prediction (for half the set, ie from 13 to 25) is, binomial, ie:

$$e_{\text{ensemble}} = \sum_{i=13}^{25} \binom{25}{i} \varepsilon^i (1-\varepsilon)^{25-i} = 0.06$$

- 6% much less than 35%

(Ref: "A Survey of Ensemble Classification - SMU", "Tan, Steinbach , and Kumar")

Ensemble Rationale

Conditions necessary for an ensemble classifier to perform better than a single classifier:

- Base classifiers should be independent of each other
- Base classifiers should not do worse than a classifier doing random guessing
- Example: for two-class problem, base classifier error rate: $\epsilon < .5$

When to Ensemble?

- Given a fixed-size number of training samples, our model will increasingly suffer from the "curse of dimensionality" if we increase the number of features.
- The challenge of individual, un-pruned decision trees is that the model often ends up being too complex for the underlying training data – decision trees are prone to over-fitting.

For Complex Decision Making

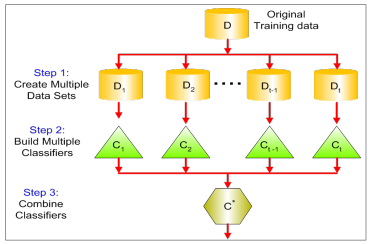
- Ensemble are called as"meta-algorithms": approaches to combine several machine learning techniques into one predictive model
- Bagging and random forests are "bagging" algorithms that aim to reduce the complexity of models that overfit the training data.
- In contrast, boosting is an approach to increase the complexity of models that suffer from high bias, that is, models that underfit the training data.

Ensemble Characteristics

- Build multiple models from the same training data by creating each model on a modified version of the training data.
- Make a final, ensemble prediction by aggregating the predictions of the individual models
- Classification prediction: Let each model have a vote on the correct class prediction. Assign the class with the most votes.
- Regression prediction: Measure of central tendency (mean or median)

Ensemble Constructing

- By manipulating the training set.
- By manipulating the input features.
- By manipulating the class labels.
- By manipulating the learning algorithm.



(Reference: Basics of Ensemble Learning Explained in Simple English - Tavish Shrivastava)

Ensemble Approaches

Decision Tree Model Ensembles

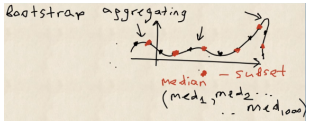
- Bagging
- Boosting
- Random Forests

Bagging (Bootstrapping)

- Stands for **B**ootstrap **Agg**regat**ing**
- It was proposed by Leo Breiman in 1994.
- A way to decrease the variance of your prediction
- By generating additional data for training from your original data-set
- Using combinations with repetitions to produce multisets of the same cardinality/size as your original data.

Statistical Idea of Bootstrapping

- Say, you have a complex distribution and you are asked to find its median.
- If it was some standard distribution like Normal, then formulae are available
- Here the trick is to sample, say randomly pick 10 points, and then find median of them.
- Do this multiple times, say , 1000. You will have an array of medians. Find its median as answer.



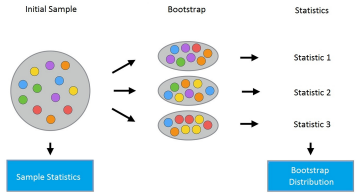
Bootstrap is used for Bagging.

Idea of Bagging

- Let there be a sample X of size N.
- We can make a new sample from the original sample by drawing N elements from the latter randomly and uniformly, with replacement.
- In other words, we select a random element from the original sample of size N and do this N times.
- All elements are equally likely to be selected, thus each element is drawn with the equal probability $\frac{1}{N}$.
- Due to 'With replacement' mode, in all cases total is always N.
- Note that, because we put the balls back, there may be duplicates in the new sample. Let's call this new sample X_1

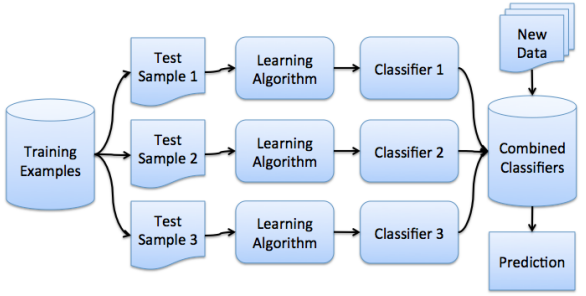
Bagging

By repeating this procedure M times, we create M bootstrap samples $X_1, \ldots, X_M$. In the end, we have a sufficient number of samples and can compute various statistics of the original distribution.



Distribution of the statistics (say, median) from each sample creates the Bootstrap distribution.
Condition for Bagging: Samples/rows must be idd (independently identically distributed)

Bagging



Bagging

- We train a number (ensemble) of decision trees from bootstrap samples of our training set.
- Bootstrap sampling means drawing random samples from our training set with replacement.
- E.g., if our training set consists of 7 training samples, our bootstrap samples

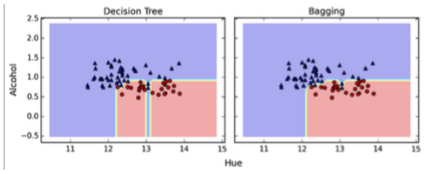
Sample indices	Bagging round 1	Bagging round 2	...
1	2	7	...
2	2	3	...
3	1	2	...
4	3	1	...
5	7	1	...
6	2	7	...
7	4	7	...
	C_1	C_2	C_m

Bagging

- After we trained our (m) decision trees, we can use them to classify new data via majority rule.
- For instance, we'd let each decision tree make a decision and predict the class label that received more votes.
- Typically, this would result in a less complex decision boundary, and the bagging classifier would have a lower variance (less overfitting) than an individual decision tree.

Bagging

- Below is a plot comparing a single decision tree (left) to a bagging classifier (right)



(Reference: https://sebastianraschka.com/faq/docs/bagging-boosting-rf.html)

Boosting

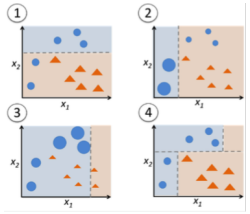
- Takes a random sample from training set
- Trains a model
- Uses original data from Training to test how good this new model is.
- For the next model, when we chose sample from the training set, the points which got mis-classified previously are weighted more to be picked up now.
- Train the next model.
- We test both models together by assembling their individual output.

Boosting

- In contrast to bagging, we use very simple classifiers as base classifiers, so-called "weak learners."
- Picture these weak learners as "decision tree stumps" – decision trees with only 1 splitting rule.

Boosting

- We start with one decision tree stump (1) and "focus" on the samples it got wrong.
- In the next round, we train another decision tree stump that attempts to get these samples right (2);
- Again, this 2nd classifier will likely get some other samples wrong, so do it one more time



Boosting

- Unlike bagging, in the classical boosting the subset creation is not random and depends upon the performance of the previous models
- Every new subsets contains the elements that were (likely to be) misclassified by previous models.

Boosting

- Boosting works by iteratively creating models and adding them to the ensemble
- Iteration stops when a predefined number of models have been added
- Each new model added to the ensemble is biased to pay more attention to instances that previous models mis-classified (weighted dataset).

Boosting Example

Boosting (Round 1)	7	3	2	8	7	9	4	10	6	3

- Suppose that *Instance #4* is hard to classify.
- Weight for this instance will be increased in future iterations, as it gets misclassified repeatedly.
- Examples not chosen in previous round (*Instances #1, #5*) also may have better chance of being selected in next round.
  - *Why?* Predictions in previous round are likely to be wrong since they weren't trained on.

Boosting (Round 2)	5	4	9	4	2	5	1	7	4	2
Boosting (Round 3)	4	4	8	10	4	5	4	6	3	4

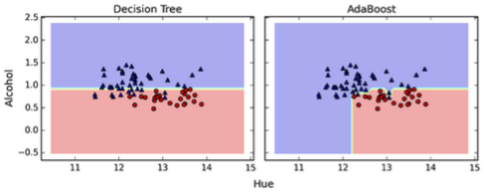
- As boosting rounds proceed, instances that are the hardest to classify become even more prevalent.

Prediction

- Once the set of models have been created the ensemble makes predictions using a weighted aggregate of the predictions made by the individual models.
- The weights used in this aggregation are simply the confidence factors associated with each model.
- Several different boosting algorithms exist
- Different by:
  - How weights of training instances are updated after each boosting round
  - How predictions made by each classifier are combined. Each boosting round produces one base classifier

AdaBoost

- AdaBoost is a popular boosting algorithm
- "Adaptive" or "incremental" learning from mistakes.
- Eventually, we will come up with a model that has a lower bias than an individual decision tree (thus, it is less likely to underfit the training data)



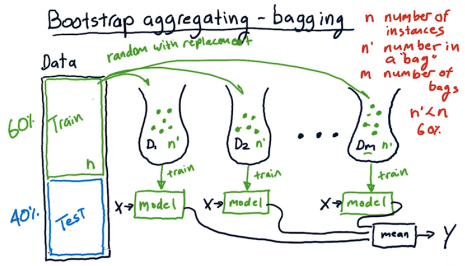
Bagging vs. Boosting

- Boosting: Sample with nonuniform distribution
- Unlike bagging where each instance had equal chance of being selected
- Boosting Motivation: focus on instances that are harder to classify
- How: give harder instances more weight in future rounds

Bagging (recap)

- Parallel ensemble: each model is built independently
- Aim to decrease variance, not bias
- Suitable for high variance low bias models (complex models)
- An example of a tree based method is random forest, which develop fully grown trees (note that RF modifies the grown procedure to reduce the correlation between trees)

Bagging (recap)

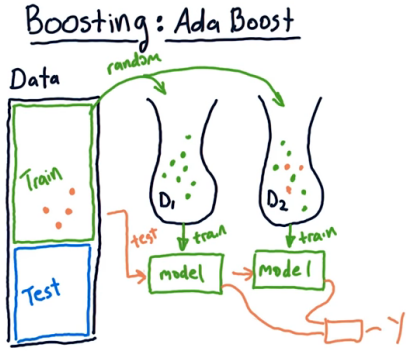


(Reference: Bootstrap aggregating bagging - Udacity)

Boosting (recap)

- Sequential ensemble: try to add new models that do well where previous models lack
- Aim to decrease bias, not variance
- Suitable for low variance high bias models
- An example of a tree based method is gradient boosting

Boosting (recap)



(Reference: Bootstrap aggregating bagging - Udacity)

Ensemble Choices

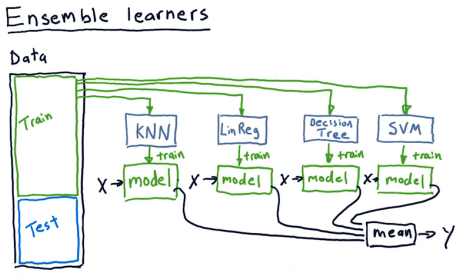
- Which approach should we use?
- Bagging is simpler to implement and parallelize than boosting and, so, may be better with respect to ease of use and training time.

Ensemble Choices

Empirical results indicate:

- Boosted decision tree ensembles were the best performing model of those tested for datasets containing up to 4,000 descriptive features.
- Random forest ensembles (based on bagging) performed better for datasets containing more that 4,000 features.

Ensemble Methods (Recap)



(Reference: Bootstrap aggregating bagging - Udacity)

Ensemble Methods (Recap)

- Advantages:
  - Astonishingly good performance
  - Modeling human behavior: making judgment based on many trusted advisors, each with their own specialty
- Disadvantage:
  - Combined models rather hard to analyze. Tens or hundreds of individual models
  - Current research: making models more comprehensible

Random Forest

Random Forests

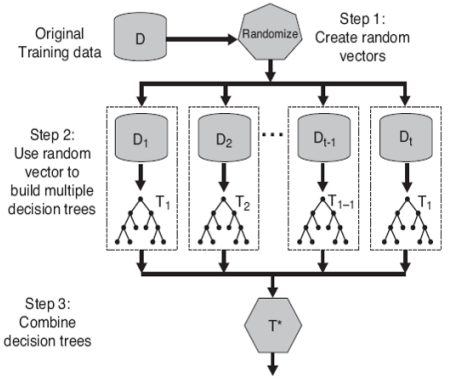
- Leo Breiman managed to apply bootstrapping not only in statistics but also in machine learning.
- He, along with Adel Cutler, extended and improved the random forest algorithm proposed by Tin Kam Ho.
- They combined the construction of uncorrelated trees using CART, bagging, and the random subspace method.
- Decision trees are a good choice for the base classifier in bagging because they are quite sophisticated and can achieve zero classification error on any sample.

See "Working with Leo Breiman on Random Forests, Adele Cutler" video at Youtube.

Random Forests

- Actually a bagging algorithm
- Also here, we draw random bootstrap samples from our training set.
- However, in addition to the bootstrap samples, we also draw random subsets of features for training the individual trees;
- In bagging, we provide each tree with the full set of features.
- Due to the random feature selection, the trees are more independent of each other compared to regular bagging, which often results in better predictive performance (due to better variance-bias trade-offs), and it's also faster than bagging, because each tree learns only from a subset of features.

Random Forest



Random Forests

- Random forests fit many decision trees to the same training data.
- First takes a random sample of features to grow each decision tree.
- Evaluate. Compare with previous trees.
- A forest is produced.
- A joint classification model is produced using all the trees.

Random Forest Algorithm

- Let the number of instances be equal to ℓ, and the number of feature dimensions be equal to d.
- Choose L as the number of individual models in the ensemble.
- For each model l, choose the number of features $dl < d$. As a rule, the same value of dl is used for all the models.
- For each model l, create a training set by selecting dl features at random from the whole set of d features.
- Train each model.
- Apply the resulting ensemble model to a new instance by combining the results from all the models in L. You can use either majority voting or aggregation of the posterior probabilities.

Random Forest Algorithm

Constructing a random forest of N trees goes as follows:

- For each $k = 1, \ldots, N$:
  - Generate a bootstrap sample X_k.
  - Build a decision tree b_k on the sample X_k:
    - Pick the best feature dimension according to the given criteria.
    - Split the sample by this feature to create a new tree level. Repeat this procedure until the sample is exhausted.
    - Building the tree until any of its leaves contains no more than n_{min} instances or until a certain depth is reached.
    - For each split, we first randomly pick m features from the d original ones and then search for the next best split only among the subset.

Random Forest Algorithm

The final classifier is defined by:
$a(x) = \frac{1}{N} \sum_{k=1}^{N} b_k(x)$

- We use the majority voting for classification and the mean for regression.
- For classification problems, it is advisable to set $m = \sqrt{d}$.
- For regression problems, we usually take $m = \frac{d}{3}$, where d is the number of features.
- It is recommended to build each tree until all of its leaves contain only $n_{\min} = 1$ examples for classification and $n_{\min} = 5$ examples for regression.
- You can see random forest as bagging of decision trees with the modification of selecting a random subset of features at each split.

Intuition
$a(x)$ is just an average (or vote) over N trees – averaging cancels out each tree's individual noise, which is why more (decorrelated) trees keep reducing variance. Restricting each split to a random subset of m features is what decorrelates the trees in the first place: without it, all N trees would keep picking the same strongest feature first and end up highly similar, and averaging similar things doesn't reduce variance nearly as much.

Task

There are 7 jurors in the courtroom. Each of them individually can correctly determine whether the defendant is guilty or not with 80% probability. How likely is the jury will make a correct verdict jointly if the decision is made by majority voting?

- 20.97%
- 80.00%
- 83.70%
- 96.66%

Answer: 96.66%. With 7 jurors at $p = 0.8$ each, the jury is correct whenever 4 or more agree: $\sum_{k=4}^{7} \binom{7}{k} (0.8)^k (0.2)^{7-k} \approx 0.9666$ – higher than any single juror's 80%, illustrating the same jury-theorem effect as the ensemble formula above.

Random Forest Splits

- Selection of random subset of features to determine each split
- How large should subset be?
  - Typically root(K), for K features works well for classification
  - (K / 3) for regression
- Tree is grown to full size with pruning
- Overall prediction is majority vote from all individually trained trees
- Different variations of Random Forests exist

Comparison with Decision Trees and Bagging

- In a random forest, the best feature for a split is selected from a random subset of the available features while, in bagging, all features are considered for the next best split.
- "Normal" decision tree induction utilizes full set of features to determine each split
- Random forest injects randomness

Pros and cons of random forests

Pros

- High prediction accuracy; will perform better than linear algorithms in most problems; the accuracy is comparable with that of boosting.
- Robust to outliers, thanks to random sampling.
- Insensitive to the scaling of features as well as any other monotonic transformations due to the random subspace selection.
- Doesn't require fine-grained parameter tuning, works quite well out-of-the-box. With tuning, it is possible to achieve a 0.5–3
- Efficient for datasets with a large number of features and classes.
- Handles both continuous and discrete variables equally well.

Pros and cons of random forests

Pros

- Rarely overfits. In practice, an increase in the tree number almost always improves the composition. But, after reaching a certain number of trees, the learning curve is very close to the asymptote.
- There are developed methods to estimate feature importance.
- Works well with missing data
- Provides means to weight classes on the whole dataset as well as for each tree sample.
- Easily parallelized and highly scalable.

Pros and cons of random forests

Cons

- In comparison with a single decision tree, Random Forest's output is more difficult to interpret.
- There are no formal p-values for feature significance estimation. Performs worse than linear methods in the case of sparse data: text inputs, bag of words, etc.
- Unlike linear regression, Random Forest is unable to extrapolate. But, this can be also regarded as an advantage because outliers do not cause extreme values in Random Forests.
- In the case of categorical variables with varying level numbers, random forests favor variables with a greater number of levels. The tree will fit more towards a feature with many levels because this gains greater accuracy.
- The resulting model is large and requires a lot of RAM.

Random Forest (Recap)

- Random Forest is an ensemble technique used for classification, regression.
- It combines the output of weaker techniques in order to get a stronger result.
- The weaker technique in this case is a decision tree.

Random Forest (Recap)

- Decision trees work by splitting the and re-splitting the data by features.
- If a decision tree is split along good features, it can give a decent predictive output.
- Random Forest works by averaging decision tree output, but it's a bit more complicated than that.

Random Forest (Recap)

- It also ranks an individual tree's output, by comparing it to the known output from the training data.
- This allows it to rank features. Some of the decision trees will perform better, and so the features within the tree will be deemed more important.